

Schematic Design of an Allwinner H3 Based Mini PC for Embedded Linux Applications

Gudisa Dinesh Reddy
MT2025509

Department of Electronics and Communication Engineering
International Institute of Information Technology Bangalore
Email: Gudisa.Reddy@iiitb.ac.in

Project Repository:

[Click Here to Access GitHub Repository](#)

Abstract—This paper presents the schematic-level design of a compact Mini PC system based on the Allwinner H3 application processor. The proposed system supports Embedded Linux booting using external DDR3 memory and non-volatile boot storage such as SPI Flash and Micro SD card. The schematic integrates the major hardware blocks required for a Linux-capable embedded computer, including processor, DDR3 RAM, boot memory, power management, USB interfaces, HDMI display output, Wi-Fi/Bluetooth connectivity, reset circuitry, and clock generation. The work is limited to schematic design because the Allwinner H3 is a BGA package device, and practical PCB layout for DDR3, HDMI, and USB requires multilayer routing, impedance control, length matching, and professional fabrication.

Index Terms—Allwinner H3, Mini PC, Embedded Linux, DDR3, SPI Flash, SD Card, Booting, Schematic Design, BGA, KiCad.

I. INTRODUCTION

A Mini PC is a compact computing system capable of running a full operating system such as Linux. Unlike a microcontroller-based embedded system, a Mini PC requires an application processor, external RAM, boot storage, regulated power rails, clock circuits, reset circuits, and high-speed peripheral interfaces.

In this project, the Allwinner H3 processor is used as the main processing unit [1] is used as the main processing unit. It interfaces with DDR3 RAM, SPI Flash, Micro SD card, USB, HDMI, and Wi-Fi/Bluetooth modules. The overall schematic architecture of the proposed system is shown in Fig. 1. The main objective of this work is to design and explain a complete schematic architecture for a Linux-capable embedded Mini PC.

The proposed system provides educational and architectural understanding of embedded Linux hardware platforms. It demonstrates the interaction between processor, memory, boot storage, power management, and communication peripherals within a modern embedded computing system.

II. MOTIVATION

Modern embedded systems increasingly require compact Linux-capable computing platforms for multimedia, networking, automation, and edge-computing applications. Designing

such systems requires understanding of high-speed memory interfaces, boot architecture, power sequencing, and peripheral integration.

The motivation behind this work is to develop a complete schematic-level understanding of a Linux-capable Mini PC platform using the Allwinner H3 processor while studying practical embedded hardware architecture, DDR3 interfacing, power management, and Linux boot mechanisms.

III. PROJECT SCOPE

The scope of this work is limited to schematic design. PCB layout is not implemented because the Allwinner H3 is a BGA package device. Designing a reliable PCB for this processor requires:

- BGA fanout routing.
- Multilayer PCB stack-up.
- DDR3 length matching.
- Controlled impedance routing for USB and HDMI.
- Proper power plane design.
- Signal integrity verification.

Therefore, this project demonstrates the complete schematic architecture and explains booting, dumping, power distribution, memory interface, and peripheral operation. The reason for restricting the work to schematic-level design is further discussed in Section XIV.

IV. SYSTEM ARCHITECTURE

The proposed Mini PC architecture consists of the Allwinner H3 processor, DDR3 memory, SPI Flash, Micro SD card, USB interfaces, HDMI output, Wi-Fi/Bluetooth module, and power management circuit.

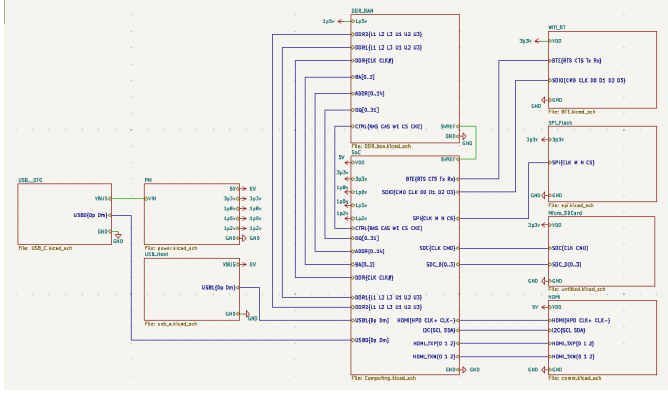


Fig. 1: Top-level schematic of the proposed Allwinner H3 based Mini PC system.

As shown in Fig. 1, the top-level schematic represents the complete system interconnection. The processor is connected to memory and peripherals through named buses such as DDR, SPI, USB, HDMI, SDIO, and SDC. This modular schematic organization improves readability and allows each functional block to be designed and verified separately.

V. ALLWINNER H3 PROCESSING ARCHITECTURE

The Allwinner H3 application processor acts as the central computational unit of the proposed Mini PC system. Unlike a conventional microcontroller, the H3 is an application processor capable of executing Embedded Linux using external DDR3 memory and non-volatile boot storage.

The processor supports multiple important interfaces such as DDR3 memory controller, USB 2.0, HDMI, SPI, SDIO, UART, I2C, and GPIO [1]. Due to the large number of pins and functional groups, the processor schematic is organized hierarchically as shown in Fig. 2.

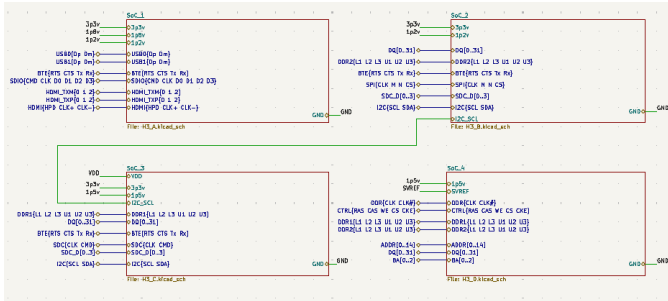


Fig. 2: Hierarchical organization of the Allwinner H3 processing subsystem.

The H3 processor is responsible for:

- Executing the internal Boot ROM after power-on.
- Detecting a valid boot device.
- Loading the Secondary Program Loader.
- Initializing external DDR3 memory.
- Loading U-Boot into DDR3 memory.
- Loading and executing the Linux kernel.
- Managing peripheral communication interfaces.

The processor communicates with external DDR3 memory using address, data, control, strobe, and differential clock signals. Peripheral interfaces such as SPI Flash, Micro SD card, USB, HDMI, and SDIO-based Wi-Fi/Bluetooth are connected through dedicated processor interfaces, as indicated in Fig. 2.

VI. DDR3 RAM

External DDR3 RAM is mandatory because Linux cannot execute only from the small internal SRAM available inside the processor. The top-level DDR3 bus organization is shown in Fig. 3.

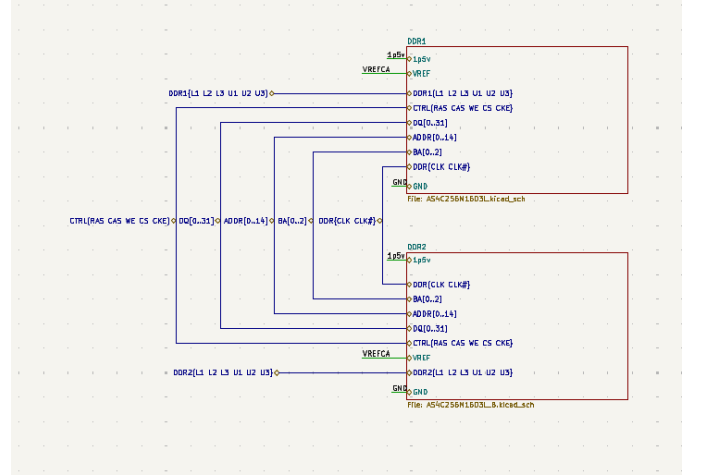


Fig. 3: DDR3 RAM top-level bus connection.

In the proposed schematic, two AS4C256M16D3L DDR3 devices are used to implement the external memory subsystem based on the memory organization supported by the H3 processor [1]. Each DDR3 device provides 512 MB capacity with a 16-bit data width. Together, the two devices provide:

$$512 \text{ MB} \times 2 = 1024 \text{ MB} = 1 \text{ GB} \quad (1)$$

The two DDR3 chips share the same address and control buses. Therefore, both devices receive identical ADDR[0..14], BA[0..2], RAS, CAS, WE, CS, CKE, and differential clock signals from the processor, as shown in Fig. 3.

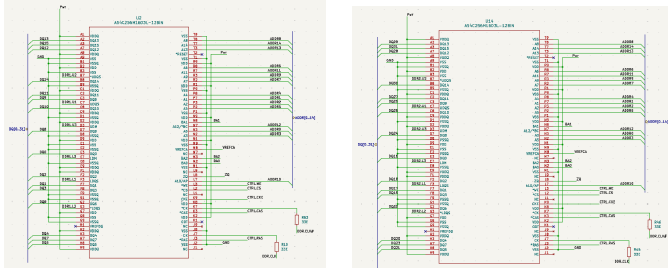
The data bus is divided into two sections:

- DDR1 handles DQ[0..15].
- DDR2 handles DQ[16..31].

This arrangement forms a complete 32-bit DDR3 memory interface for the H3 processor while combining the capacities of both devices to achieve a total memory size of 1 GB. The two DDR3 memory devices are shown in Fig. 4.

The DDR interface consists of address buses, bank address buses, data buses, differential clock signals, data strobes, and control signals required for high-speed memory communication.

DDR3 uses differential clocking because high-speed memory operation requires accurate timing, improved signal integrity, and reduced noise sensitivity. During booting, DDR3 is initialized by the SPL before larger software components such



(a) First 512 MB DDR3. (b) Second 512 MB DDR3.

Fig. 4: Two DDR3 devices forming total 1 GB memory.

as U-Boot and the Linux kernel are loaded, as summarized later in Table II.

VII. BOOT STORAGE

The proposed design supports two boot storage options: SPI Flash and Micro SD card. These memories store the bootloader and operating system files permanently.

A. SPI Flash

The SPI Flash schematic is shown in Fig. 5. It operates as non-volatile memory and retains bootloader data even after power is removed. During startup, the Boot ROM reads the initial bootloader stages from SPI Flash before transferring execution into DDR3 memory.

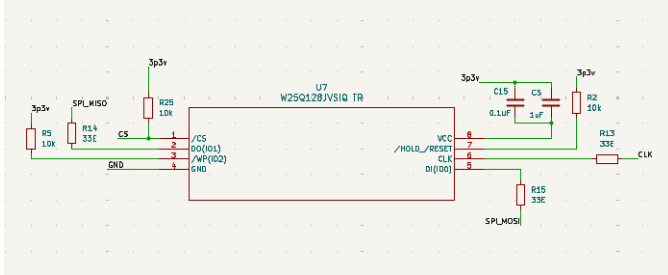


Fig. 5: SPI Flash schematic using W25Q128JVS1Q.

The SPI Flash is connected to the H3 through SPI clock, MOSI, MISO, and chip select signals. It can store the SPL and U-Boot bootloader. However, SPI Flash usually has limited capacity compared to SD card or eMMC. Therefore, it is more suitable for storing bootloader stages rather than a complete Linux root file system.

B. Micro SD Card

The Micro SD card interface is shown in Fig. 6. It provides a flexible boot solution because Linux images can be easily updated or replaced directly from a host PC without requiring dedicated programming hardware.

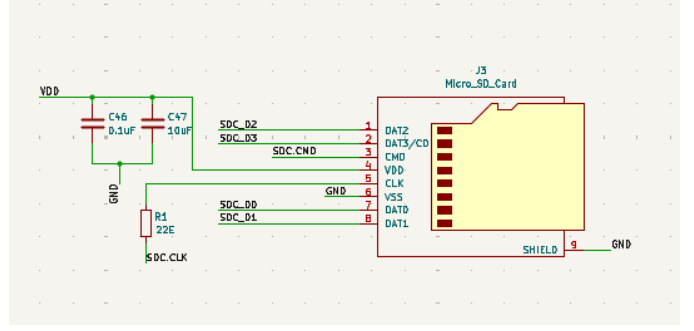


Fig. 6: Micro SD card boot storage schematic.

The Micro SD card interface uses CMD, CLK, and four data lines SDC_D[0..3]. The SD card can store SPL, U-Boot, Linux kernel, device tree, and root file system. For development and academic demonstration, SD card booting is the most convenient method because the Linux image can be written directly from a PC.

VIII. POWER MANAGEMENT

The H3 processor, DDR3 memory, and peripheral interfaces require multiple regulated voltage rails. Different internal blocks operate at different voltage levels; therefore, separate power domains are necessary for stable operation. The power management schematic is shown in Fig. 7, and the voltage rails are summarized in Table I.

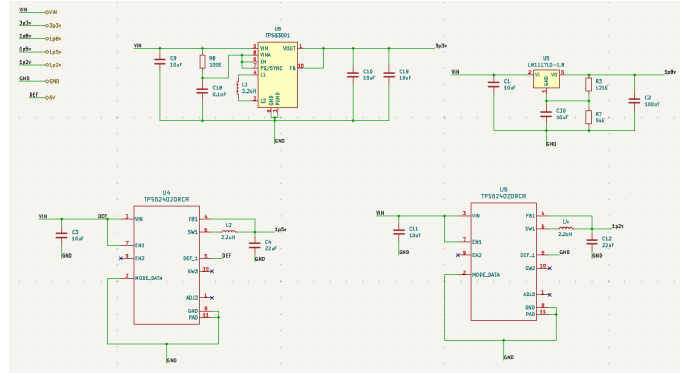


Fig. 7: Power management section generating required voltage rails.

TABLE I: Power Rails Used in the Proposed Design

Voltage Rail	Purpose
5 V	Main input supply and HDMI connector supply
3.3 V	I/O supply, SPI Flash, SD card, Wi-Fi/Bluetooth module
1.8 V	PLL and internal interface supplies
1.5 V	DDR3 memory supply
1.2 V	Processor core supply

As listed in Table I, separate voltage rails are required for the processor core, I/O interfaces, DDR3 memory, and peripheral circuits. Decoupling capacitors are placed near power pins to reduce voltage fluctuations and provide local

transient current during switching operations. Bulk capacitors are used near regulator outputs to improve supply stability.

Proper power sequencing is important in Linux-capable systems because incorrect voltage initialization may prevent DDR3 initialization or processor booting. Separate voltage rails also help reduce electrical noise between processor core, DDR memory, and peripheral interfaces.

IX. USB INTERFACES

The design includes USB-C based OTG and USB host interfaces. USB OTG is useful for development, recovery, and possible FEL mode communication. USB host can be used for peripherals such as keyboard, mouse, or USB storage. The USB-C OTG and USB host schematics are shown in Fig. 8 and Fig. 9, respectively.

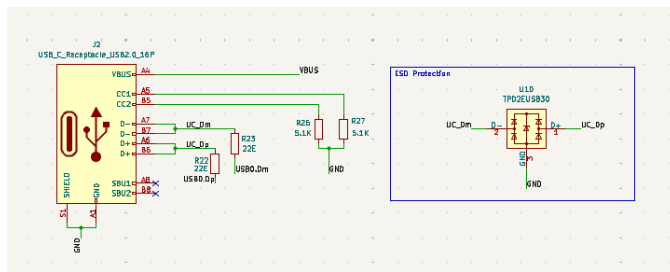


Fig. 8: USB-C OTG interface schematic.

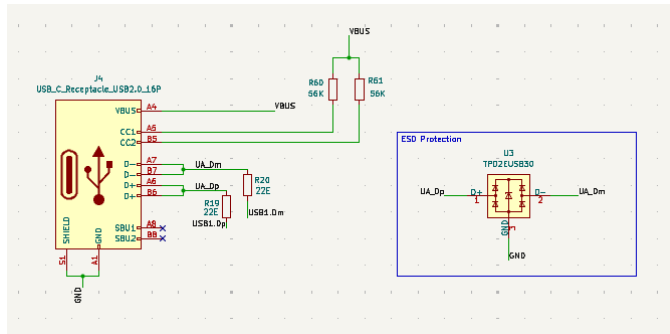


Fig. 9: USB host interface with ESD protection.

USB D+ and D- are differential signals. In a practical PCB layout, these lines require controlled impedance routing, short trace length, and proper ESD protection.

As shown in Fig. 8 and Fig. 9, ESD protection devices are included in the USB interfaces to protect the processor from electrostatic discharge events that may occur during cable insertion or removal.

X. HDMI INTERFACE

The HDMI interface provides display output from the Mini PC to an external monitor. The HDMI connector and protection circuit are shown in Fig. 10.

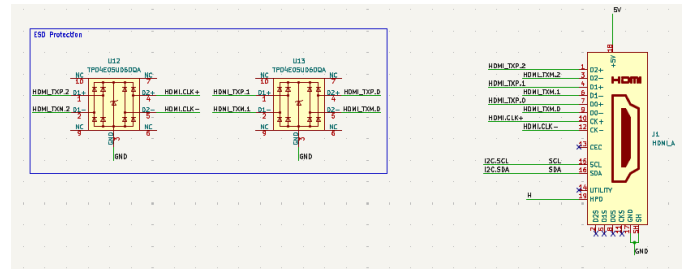


Fig. 10: HDMI connector and ESD protection circuit.

As shown in Fig. 10, the HDMI section contains TMDS differential data pairs, TMDS clock pair, DDC lines SCL and SDA, Hot Plug Detect, 5 V HDMI supply, and ESD protection. The HDMI 5 V line is provided on the connector side for display detection, while logic-level signals must remain compatible with the processor voltage domain.

The HDMI interface uses TMDS differential signaling to support high-speed digital video transmission with improved noise immunity and signal integrity.

XI. WI-FI AND BLUETOOTH SECTION

The wireless connectivity block is implemented using the RTL8723DS-CG Wi-Fi/Bluetooth module, as shown in Fig. 11.

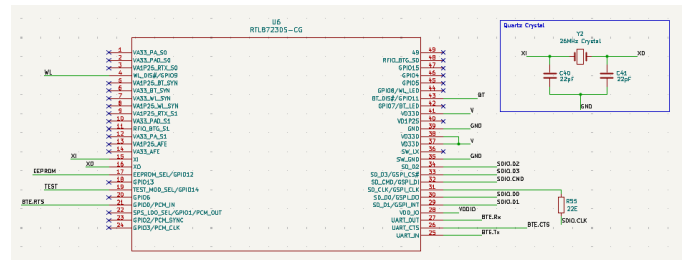


Fig. 11: RTL8723DS Wi-Fi/Bluetooth module schematic.

Wi-Fi communication uses the SDIO interface, which includes SDIO clock, command, and data lines. Bluetooth communication uses UART signals such as TX, RX, RTS, and CTS. RTS and CTS are hardware flow control signals that help prevent data loss during communication.

The SDIO interface provides higher data throughput compared to standard UART communication and is therefore suitable for wireless networking applications requiring continuous high-speed data transfer. Control pins such as WL_DIS and BT_DIS are used to enable or disable Wi-Fi and Bluetooth sections respectively.

XII. DUMPING OR PROGRAMMING PROCESS

Dumping refers to writing the bootloader, Linux kernel, device tree, and root file system into the selected boot storage device. In this project, dumping can be performed using either SD card programming or SPI Flash programming. The SPI Flash and SD card boot-storage circuits are shown earlier in Fig. 5 and Fig. 6.

TABLE II: Linux Boot Sequence of the Proposed System

Stage	Operation
1	Power rails stabilize and reset is released
2	Internal Boot ROM starts execution
3	Boot device detection is performed
4	SPL is loaded into internal SRAM
5	DDR3 memory initialization is completed
6	U-Boot is loaded into DDR3 memory
7	Linux kernel and device tree are loaded
8	Linux kernel execution begins

A. SD Card Dumping

The SD card dumping method is the simplest and most practical method for development.

- 1) Build or download an Allwinner H3 compatible Linux image.
- 2) Insert the Micro SD card into a PC using a card reader.
- 3) Write the image using Balena Etcher, Win32 Disk Imager, or the Linux `dd` command.
- 4) Insert the programmed SD card into the board.
- 5) Power on the board.
- 6) The H3 Boot ROM detects the SD card and starts the boot process.

The SD card image contains SPL, U-Boot, Linux kernel, device tree, and root file system.

B. SPI Flash Dumping

SPI Flash can be programmed using USB FEL mode supported by the internal Boot ROM mechanism of Allwinner processors [3].

- 1) Connect the board to a PC through USB OTG.
- 2) Enter FEL or recovery mode.
- 3) Load a temporary program into internal SRAM.
- 4) Initialize DDR3 memory.
- 5) Write SPL and U-Boot into SPI Flash.
- 6) Reset the board.
- 7) The H3 Boot ROM loads the bootloader from SPI Flash.

SPI Flash is useful mainly for bootloader storage. For a complete Linux system, SD card or eMMC is preferred because the Linux root file system requires larger storage capacity.

The dumping process allows the operating system image, bootloader, and root filesystem to be programmed into non-volatile storage devices before normal system startup.

XIII. BOOTING PROCESS

Booting is the sequence through which the processor starts execution after power-on and finally runs the Linux operating system. The operating system is permanently stored in SD card or Flash memory, but it executes from DDR3 RAM after loading. The boot sequence is summarized in Table II.

As shown in Table II, DDR3 initialization occurs before U-Boot and the Linux kernel are loaded. The complete boot process is summarized as follows:

- 1) Power is applied to the board.
- 2) Voltage regulators generate all required power rails.

- 3) Reset is released after power stabilization.
- 4) The H3 internal Boot ROM begins execution.
- 5) The Boot ROM searches for a valid boot device such as SPI Flash or SD card.
- 6) SPL is loaded into internal SRAM.
- 7) SPL initializes external DDR3 memory.
- 8) U-Boot bootloader is loaded into DDR3 RAM as part of the standard Embedded Linux boot flow [4].
- 9) U-Boot loads the Linux kernel and device tree.
- 10) Linux kernel execution starts from DDR3 memory.

The internal Boot ROM is permanently programmed by the manufacturer and follows the boot sequence defined for Allwinner SoCs [3]. Its primary function is to detect a valid boot source and load the initial bootloader into internal SRAM for further system initialization.

This process clearly demonstrates the difference between storage memory and runtime memory. Flash or SD card stores the operating system permanently, while DDR3 temporarily stores the operating system during execution.

XIV. REASON FOR SCHEMATIC-ONLY DESIGN

The PCB layout is not implemented because the Allwinner H3 processor is a BGA package device with multiple high-speed interfaces requiring controlled PCB routing techniques [5] and the design contains multiple high-speed interfaces. The need for high-speed DDR3, HDMI, and USB routing is visible from the memory, display, and USB sections shown in Fig. 4, Fig. 10, Fig. 8, and Fig. 9. A practical layout requires:

- Multilayer PCB design.
- BGA escape routing.
- DDR3 address, data, and strobe length matching.
- Controlled impedance routing for USB and HDMI.
- Continuous return paths for high-speed signals.
- Power integrity and signal integrity analysis.
- Professional PCB fabrication capability.

If these conditions are not satisfied, the board may fail to boot or may behave unreliably. Therefore, the present work focuses on schematic design, which is suitable for architectural understanding and academic demonstration.

XV. DESIGN CHALLENGES

Designing a Linux-capable embedded platform involves several hardware challenges due to high-speed interfaces and multiple voltage domains. These challenges are reflected in the DDR subsystem of Fig. 3, the HDMI interface of Fig. 10, the USB interfaces of Fig. 8 and Fig. 9, and the power rails summarized in Table I.

The major challenges identified in this project are:

- DDR3 memory routing and timing constraints.
- Differential pair routing for HDMI and USB.
- BGA escape routing for the H3 processor.
- Stable multi-rail power generation.
- Signal integrity and impedance control.
- Correct boot sequence and DDR initialization.

Although this work focuses only on schematic implementation, the project demonstrates the architectural understanding necessary for practical high-speed embedded hardware design.

XVI. APPLICATIONS

The proposed Mini PC architecture shown in Fig. 1 can be used in several embedded and computing applications:

- Embedded Linux learning platform.
- IoT gateway.
- Multimedia display system.
- Lightweight edge-computing node.
- Industrial monitoring system.
- Network-connected automation controller.
- Educational platform for processor, memory, and boot architecture.

XVII. FUTURE SCOPE

The present work focuses on schematic-level implementation and architectural understanding. In future work, the design can be extended towards multilayer PCB implementation with controlled impedance routing, DDR3 length matching, thermal analysis, and practical Linux bring-up. Additional peripherals such as Ethernet, audio codec, camera interface, and eMMC storage can also be integrated to develop a fully functional embedded computing platform.

XVIII. CONCLUSION

This work presents the complete schematic-level design of an Allwinner H3 based Mini PC capable of running Embedded Linux. As shown in Fig. 1, the proposed architecture integrates

1 GB DDR3 memory, SPI Flash, Micro SD card boot storage, USB interfaces, HDMI display output, Wi-Fi/Bluetooth connectivity, and dedicated power management circuitry.

The report explains the Linux boot sequence beginning from Boot ROM execution up to Linux kernel initialization in DDR3 memory, as summarized in Table II. In addition, memory organization, boot storage architecture, dumping process, voltage domains, and peripheral communication interfaces are discussed in detail.

The project also demonstrates practical understanding of processor-memory interaction, Linux boot architecture, voltage domain separation, and high-speed peripheral integration within embedded computing systems.

Although PCB layout implementation is beyond the scope of this work due to BGA routing complexity and high-speed signal requirements, the developed schematic provides a strong architectural foundation for future multilayer PCB realization and embedded Linux hardware development.

REFERENCES

- [1] Allwinner Technology, "Allwinner H3 Datasheet V1.0." Available: https://dl.linux-sunxi.org/H3/Allwinner_H3_Datasheet_V1.0.pdf
- [2] Allwinner Technology, "Quad-Core H.265/HEVC 4K OTT Box Solution." Available: <https://www.allwinnertech.com/uploads/pdf/201608011519593c.pdf>
- [3] U-Boot Documentation, "Allwinner SoC Based Boards." Available: <https://docs.u-boot.org/en/latest/board/allwinner/sunxi.html>
- [4] U-Boot Documentation. Available: <https://docs.u-boot.org/>
- [5] "Orange Pi One PC ." Available: <http://www.orangepi.org/html/hardWare/computerAndMicrocontrollers/details/Orange-Pi-One.html>